

HOMEWORK

DECORATOR DESIGN PATTERN

Rita Pinto

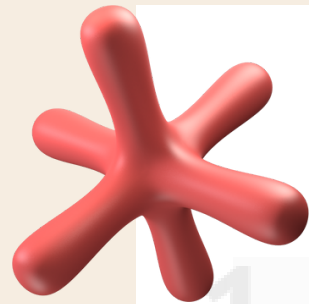


27-02-2023

Intent

The decorator design pattern is a structural design pattern that allows behavior to be added to an individual object without changing the behavior of other objects from the same class.

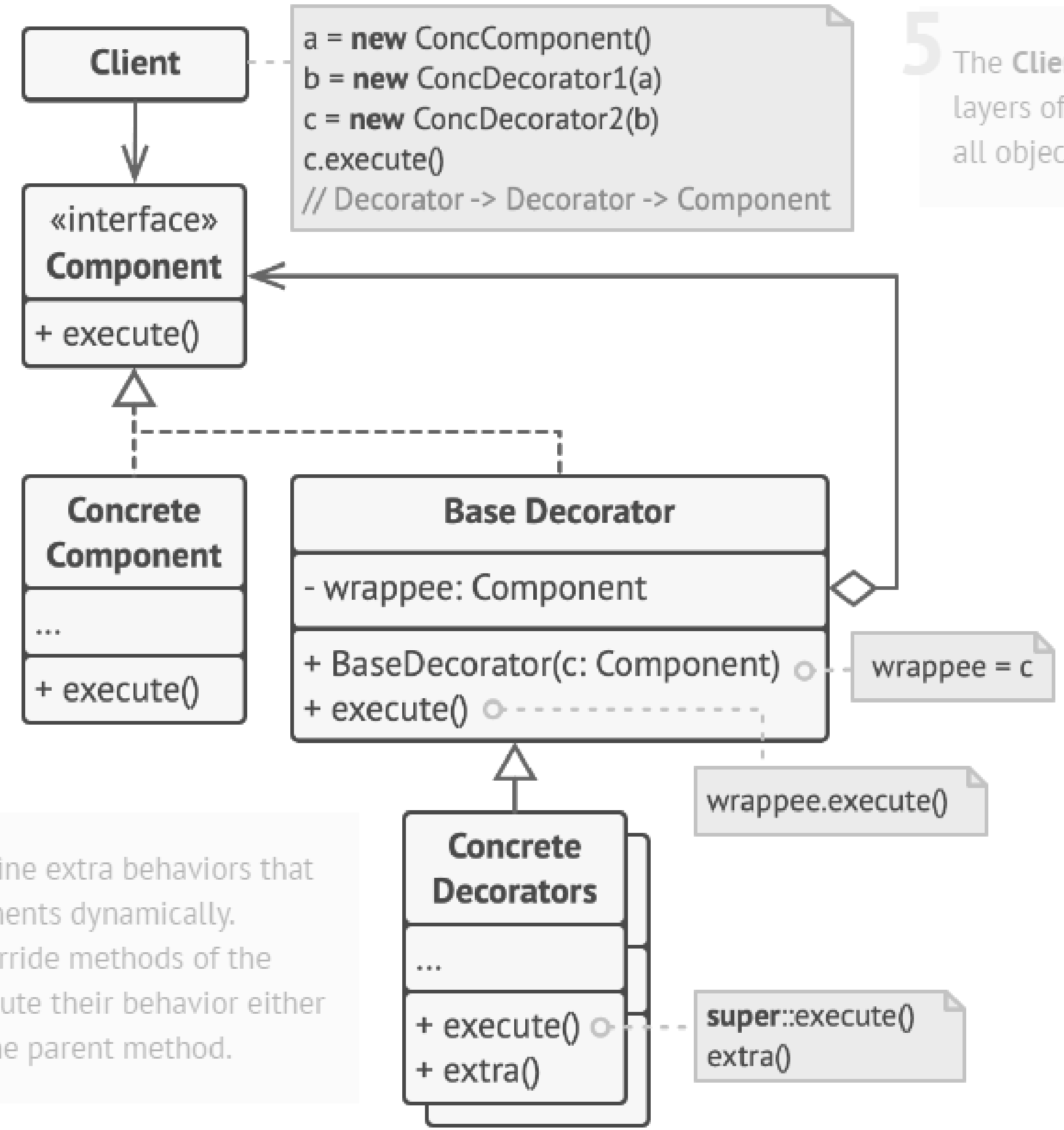




1 The **Component** declares the common interface for both wrappers and wrapped objects.

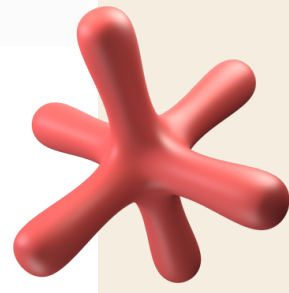
2 **Concrete Component** is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.

4 **Concrete Decorators** define extra behaviors that can be added to components dynamically. Concrete decorators override methods of the base decorator and execute their behavior either before or after calling the parent method.



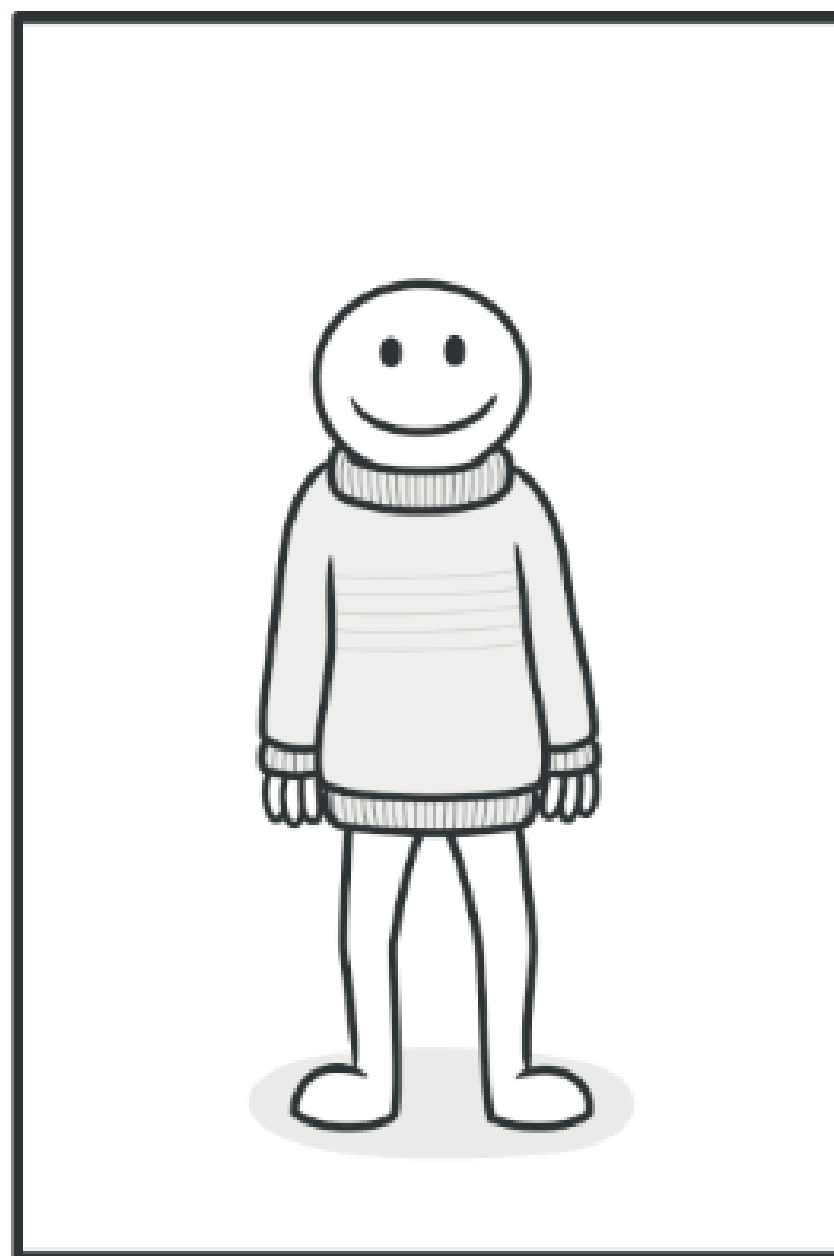
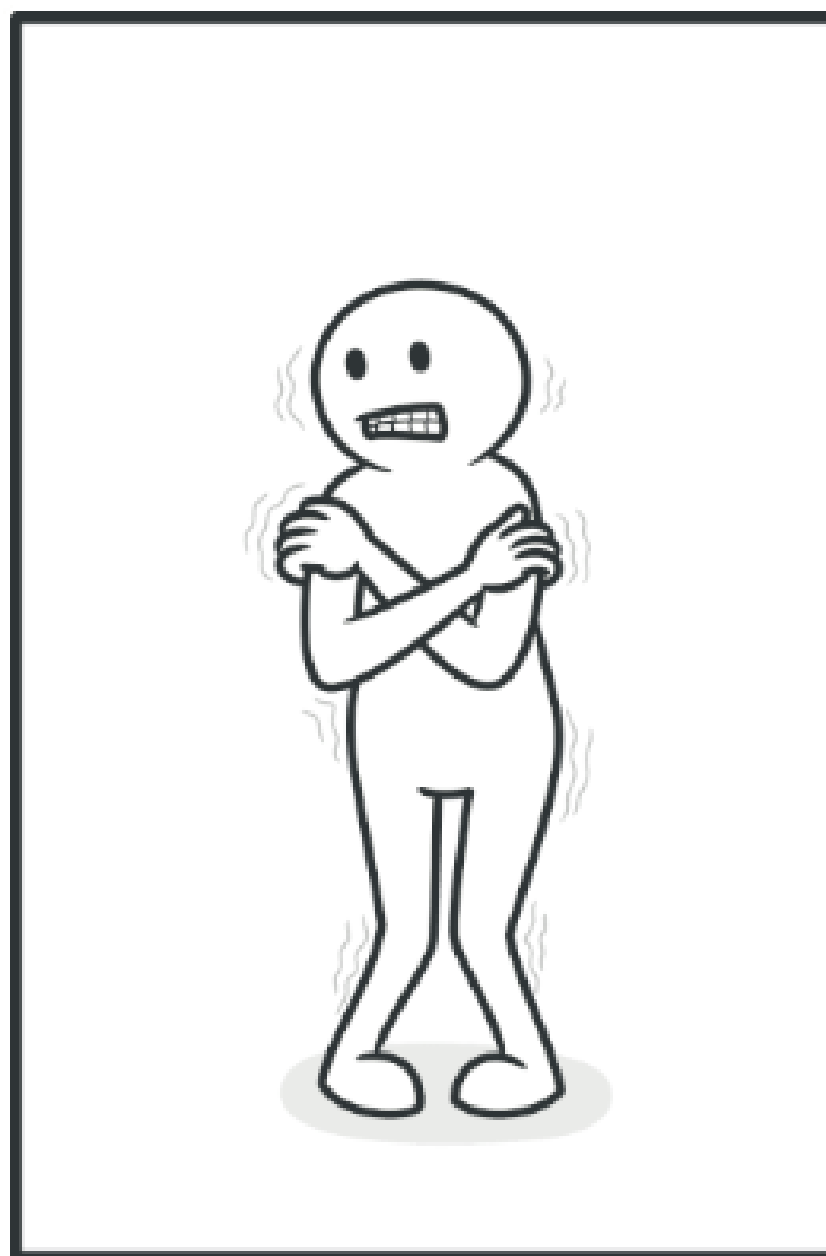
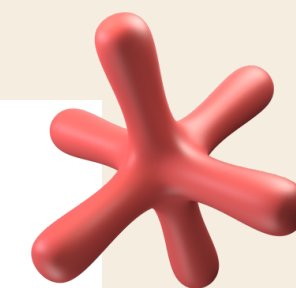
5 The **Client** can wrap components in multiple layers of decorators, as long as it works with all objects via the component interface.

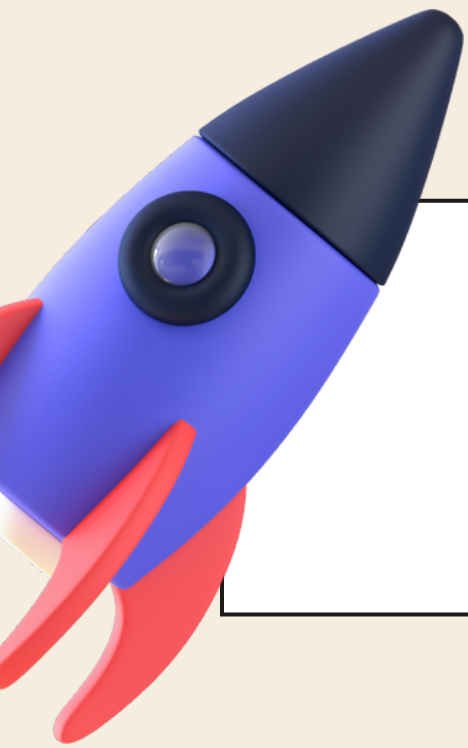
3 The **Base Decorator** class has a field for referencing a wrapped object. The field's type should be declared as the component interface so it can contain both concrete components and decorators. The base decorator delegates all operations to the wrapped object.





Real-World Analogy





PROS AND CONS

You can add or remove responsibilities from an object at runtime

The decorator design pattern can add complexity to the codebase

You can combine several behaviors by wrapping an object into multiple decorators



THANK YOU*
